

Road Accident Data Project

MySQL Database Setup, CSV Import, Data Cleaning, and Power BI Integration Guide

This report documents the complete end-to-end workflow for the Road Accident Data project: creating the MySQL database and table schema, importing the raw CSV export, cleaning and validating the data directly in MySQL, and finally connecting MySQL to Power BI for reporting and visualization.

Source File	Road Accident Data.xlsx / .csv
Target Database	road_accidents
Target Table	accident_data
Key Columns	Accident_Index, Accident_Date, Severity, Lat/Long, Vehicle_Type
Destination Tool	Power BI Desktop

Table of Contents

- 1. Creating the Database and Table in MySQL
- 2. Importing Data from CSV
- 3. Data Cleaning in MySQL
- 4. Connecting MySQL to Power BI
- 5. Summary Checklist

1. Creating the Database and Table in MySQL

The first step is to create a dedicated database for the project and define a table schema that matches the structure of the Road Accident Data export (21 columns, including identifiers, dates, categorical conditions, and numeric measures).

1.1 Create the Database

```
CREATE DATABASE road_accidents;  
USE road_accidents;
```

1.2 Create the Table

Define column types that match each field's data: text identifiers as VARCHAR, the date and time fields as DATE and TIME, and numeric measures as INT or DECIMAL.

```
CREATE TABLE accident_data (  
  Accident_Index VARCHAR(20) PRIMARY KEY,  
  Accident_Date DATE,  
  Day_of_Week VARCHAR(15),  
  Junction_Control VARCHAR(50),  
  Junction_Detail VARCHAR(50),  
  Accident_Severity VARCHAR(20),  
  Latitude DECIMAL(10,6),  
  Light_Conditions VARCHAR(50),  
  Local_Authority_District VARCHAR(100),  
  Carriageway_Hazards VARCHAR(50),  
  Longitude DECIMAL(10,6),  
  Number_of_Casualties INT,  
  Number_of_Vehicles INT,  
  Police_Force VARCHAR(50),  
  Road_Surface_Conditions VARCHAR(50),  
  Road_Type VARCHAR(50),  
  Speed_limit INT,  
  Time TIME,  
  Urban_or_Rural_Area VARCHAR(20),  
  Weather_Conditions VARCHAR(50),  
  Vehicle_Type VARCHAR(50)  
);
```

1.3 Verify the Table Structure

```
DESCRIBE accident_data;
```

Note: Accident_Index is set as the PRIMARY KEY because each value is unique per accident record. This also speeds up lookups and prevents duplicate rows on import.

2. Importing Data from CSV

Once the table exists, the next step is loading the CSV export into it. Export the Google Sheet as a CSV file first (File → Download → Comma Separated Values), then load it using one of the two methods below.

2.1 Method A: LOAD DATA INFILE (Command Line / Workbench SQL)

This is the fastest method for large files. The date column is staged into a variable and converted explicitly, since the source format (MM/DD/YYYY) does not match MySQL's native DATE format.

```
-- Enable local file loading if disabled (requires admin rights)
SET GLOBAL local_infile = 1;

LOAD DATA LOCAL INFILE '/path/to/RoadAccidentData.csv'
INTO TABLE accident_data
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(Accident_Index, @date_col, Day_of_Week, Junction_Control, Junction_Detail,
Accident_Severity, Latitude, Light_Conditions, Local_Authority_District,
Carriageway_Hazards, Longitude, Number_of_Casualties, Number_of_Vehicles,
Police_Force, Road_Surface_Conditions, Road_Type, Speed_limit, Time,
Urban_or_Rural_Area, Weather_Conditions, Vehicle_Type)
SET Accident_Date = STR_TO_DATE(@date_col, '%m/%d/%Y');
```

2.2 Method B: MySQL Workbench Table Data Import Wizard

If local_infile permissions are restricted, use Workbench's GUI importer instead:

- Right-click the **accident_data** table in the Navigator panel and choose **Table Data Import Wizard**.
- Select the CSV file and confirm the encoding (UTF-8).
- Map each CSV column to its matching table column.
- Run the import and check the log for row errors or skipped records.

2.3 Confirm the Import

```
SELECT COUNT(*) AS total_rows FROM accident_data;
SELECT * FROM accident_data LIMIT 10;
```

3. Data Cleaning in MySQL

Before connecting to Power BI, clean the data directly in MySQL so the model layer stays simple. Below are the most relevant checks and fixes for this dataset.

3.1 Check for Duplicate Records

```
SELECT Accident_Index, COUNT(*) AS cnt
FROM accident_data
GROUP BY Accident_Index
HAVING cnt > 1;
```

3.2 Check for Missing or Null Values

```
SELECT
SUM(Accident_Date IS NULL) AS missing_date,
SUM(Latitude IS NULL) AS missing_lat,
SUM(Longitude IS NULL) AS missing_long,
SUM(Speed_limit IS NULL) AS missing_speed,
SUM(Vehicle_Type IS NULL OR Vehicle_Type = '') AS missing_vehicle
FROM accident_data;
```

3.3 Standardize Text Categories

Categorical text fields like Weather_Conditions and Road_Surface_Conditions are sensitive to inconsistent casing or stray whitespace. Trim and standardize them:

```
UPDATE accident_data
SET Weather_Conditions = TRIM(Weather_Conditions),
Road_Surface_Conditions = TRIM(Road_Surface_Conditions),
Light_Conditions = TRIM(Light_Conditions),
Junction_Detail = TRIM(Junction_Detail),
Junction_Control = TRIM(Junction_Control);
```

3.4 Handle the 'Data missing or out of range' Placeholder

The Junction_Control field uses the literal text “Data missing or out of range” as a placeholder. Convert it to a proper NULL so it doesn't get counted as a real category in Power BI:

```
UPDATE accident_data
SET Junction_Control = NULL
WHERE Junction_Control = 'Data missing or out of range';
```

3.5 Validate Numeric Ranges

```
-- Latitude/Longitude should fall within UK bounds
SELECT * FROM accident_data
WHERE Latitude NOT BETWEEN 49 AND 61
OR Longitude NOT BETWEEN -8 AND 2;

-- Speed limit should be a standard UK value
SELECT DISTINCT Speed_limit FROM accident_data ORDER BY Speed_limit;

-- Casualties/vehicles should never be zero or negative
SELECT * FROM accident_data
```

```
WHERE Number_of_Casualties <= 0 OR Number_of_Vehicles <= 0;
```

3.6 Remove Exact Duplicate Rows (if any)

```
DELETE a1 FROM accident_data a1  
INNER JOIN accident_data a2  
WHERE a1.Accident_Index = a2.Accident_Index  
AND a1.Accident_Index > a2.Accident_Index;
```

Tip: Run each SELECT-based check first and review the results before running any UPDATE or DELETE statement. Consider taking a backup with mysqldump before applying destructive changes.

4. Connecting MySQL to Power BI

4.1 Install Prerequisites

- Install **MySQL Connector/NET** on the machine running Power BI Desktop — Power BI's MySQL connector depends on this driver to communicate with the database.
- Confirm the MySQL server is reachable (host, port 3306 by default, and that remote connections are allowed if Power BI is on a different machine).

4.2 Connect in Power BI Desktop

- Open Power BI Desktop and select **Get Data** → **More...**
- Search for and select **MySQL database**, then click **Connect**.
- Enter the **Server** (e.g. localhost:3306) and **Database** (road_accidents).
- Choose a connectivity mode: **Import** (loads data into Power BI's in-memory model — recommended for this dataset size) or **DirectQuery** (queries MySQL live, useful for very large or frequently changing data).
- Enter MySQL credentials (username/password) when prompted, and select **Connect**.

4.3 Select and Load the Table

In the Navigator window, check the **accident_data** table and click **Transform Data** (recommended) rather than **Load** directly — this opens Power Query, where remaining type corrections can be applied before the data enters the model.

4.4 Fix Data Types in Power Query

Even after cleaning in MySQL, verify these columns import with the correct type and correct any mismatches using the column header type-selector:

Column	Expected Type
Accident_Date	Date
Time	Time
Number_of_Casualties / Number_of_Vehicles	Whole Number
Speed_limit	Whole Number
Latitude / Longitude	Decimal Number

4.5 Click Close & Apply

Once types are confirmed, click **Close & Apply** to load the data into the Power BI model. From here, build out a star schema (date, location, conditions, junction, and vehicle dimension tables) and create visuals such as severity by weather, accident hotspot maps, and monthly trend lines.

4.6 Refreshing the Data

- **Import mode:** use Power BI Desktop's Refresh button, or schedule refreshes via Power BI Service if published, with a gateway configured for the on-premises MySQL server.
- **DirectQuery mode:** data is always current, but report performance depends on MySQL query speed.

5. Summary Checklist

Step	Task	Done
1	Create database and table schema in MySQL	■
2	Export Google Sheet to CSV	■
3	Import CSV via LOAD DATA INFILE or Workbench Wizard	■
4	Verify row count and sample data	■
5	Check and remove duplicate Accident_Index values	■
6	Check for NULL/missing values in key columns	■
7	Trim and standardize categorical text fields	■
8	Convert placeholder text to NULL where appropriate	■
9	Validate numeric ranges (lat/long, speed, counts)	■
10	Install MySQL Connector/NET	■
11	Connect Power BI to MySQL database	■
12	Choose Import or DirectQuery mode	■
13	Fix data types in Power Query	■
14	Build star schema and visuals in Power BI	■